

GitHub 活用セミナー

チーム開発のセキュリティと運用

田中博悠 / 湘南藤沢高専学生会副会長

アジェンダ

1. Git とは何か（概念）
2. なぜチーム開発で Git が必要か
3. ブランチ戦略
4. `.gitignore` とシークレット管理
5. Branch Protection Rules
6. PR とコードレビュー
7. Sign Commit
8. 脆弱性を見つけたら

1. Git とは何か

バージョン管理システム

- ファイルの変更履歴をすべて記録する仕組み
- いつ・誰が・何を変えたかを追跡できる
- 過去の任意の状態に戻せる

主要な概念

用語	意味
commit	変更を記録するスナップショット
push	ローカルの変更をリモートへ送る
pull	リモートの変更をローカルへ取り込む
clone	リモートリポジトリをローカルへ複製

ローカルとリモート

[あなたのPC]		[GitHub]
ローカルリポジトリ	↔	リモートリポジトリ
作業する場所	push/pull	共有される場所

- ローカル：自分のPCの中にある作業スペース
- リモート：GitHubなどのサーバー上にある共有リポジトリ
- オフラインでも作業できて、あとでpushできる

2. なぜチーム開発で Git が必要か

上書き事故がなくなる

❌ 最終版.zip 最終版_修正.zip 最終版_本当に最後.zip

→ Git があればこういう管理は不要

変更履歴が追える

- 「誰がこのバグを入れた？」がわかる (`git blame`)
- 「なぜこう書いた？」はコミットメッセージで残せる

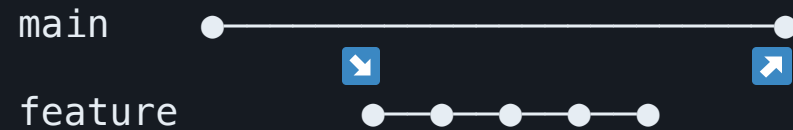
並行作業ができる

- 複数人が同時に別々の機能を開発できる
- 最終的に安全にマージできる

3. ブランチ戦略

ブランチとは

コードの分岐した作業ライン。本流（main）を壊さずに作業できる。



鉄則

❌ main ブランチに直接 push しない

GitHub Flow

シンプルで多くのチームで使われているブランチ戦略。

1. main から feature ブランチを切る
2. ブランチ上で作業・commit
3. Pull Request を作成
4. コードレビューを受ける
5. 承認されたら main へ merge
6. ブランチを削除

ポイント

- `main` は常にデプロイ可能な状態を保つ
- ブランチ名は何をするか明確に（例: `feature/add-login`, `fix/typo-readme`）

4. `.gitignore` とシークレット管理

push してはいけないもの

 絶対にリポジトリに入れてはいけない

- APIキー・アクセストークン
- `.env` ファイル
- 秘密鍵 (`*.pem` , `*.key`)
- パスワードを含む設定ファイル

`.gitignore` の書き方

`.gitignore` ファイルに書いたパターンは Git が無視する。

```
# 環境変数
.env
.env.local
.env.*.local

# 秘密鍵
*.pem
*.key

# 依存関係（大量ファイルをpushしない）
node_modules/
__pycache__/

# IDEの設定ファイル
.vscode/
.idea/
```

やらかし事例

よくある実際のインシデント

⚠ AWSのアクセスキーをうっかりGitHubにpush

→ Botが数分で検知

→ 数時間で数十万円の不正利用

- GitHub には **Secret Scanning** 機能があり、既知のトークン形式を検知してくれる
- ただし検知される前に既にBot収集済みのことが多い
- push してから消しても遅い（履歴に残る）

→ 最初からpushしないのが唯一の正解

5. Branch Protection Rules

「ルールを決める」だけでは足りない

人間はミスをする。GitHubの機能で強制できる。

設定できること

設定	効果
Require pull request reviews	PR なしの merge を禁止
Require status checks	CI 通過しないと merge 不可
Restrict who can push	直 push できる人を制限
Require signed commits	署名なし commit を拒否

Settings → Branches → Add branch ruleset から設定

6. PR とコードレビュー

Pull Request とは

「この変更を main に取り込んでください」という提案 + 議論の場

PR を出すときのポイント

- 小さく出す：レビューしやすいサイズに分ける
- 説明を書く：何を・なぜ変えたか
- スクリーンショット：UIの変更があれば添付

良いPRの説明テンプレート

概要

ログイン機能にバリデーションを追加した。

変更内容

- メールアドレスの形式チェックを追加
- 空白送信を防止

確認方法

1. ログインページへ移動
2. 不正なメールアドレスを入力
3. エラーメッセージが表示されることを確認

関連 Issue

closes #42

コードレビューとは

目的

- バグの早期発見
- セキュリティ上の問題の指摘
- 知識の共有・チームの品質統一

レビュアーとして

- 「なぜこう書いたか」を理解しようとする
- 指摘はコードに対して行う（人格攻撃ではない）
- 良い点も伝える

レビューを受ける側として

- 指摘は改善のチャンス
- 説明できない実装は疑う

7. Sign Commit

なぜ必要か

Git の `user.name` と `user.email` は自由に設定できる。

```
git config user.email "someone@example.com"  
# 他人になりすましてcommitできてしまう
```

→ コミットログは改ざん・なりすましが可能

Sign Commit とは

GPGキーで署名することで「本当に自分が書いた」を証明する。

GitHubでは署名済みコミットに `Verified` バッジが付く 

Sign Commit の設定手順

```
# 1. GPGキーを生成
gpg --full-generate-key

# 2. キーIDを確認
gpg --list-secret-keys --keyid-format=long

# 3. GitにGPGキーを設定
git config --global user.signingkey <KEY_ID>

# 4. 常に署名する設定
git config --global commit.gpgsign true

# 5. 公開鍵をGitHubに登録
gpg --armor --export <KEY_ID>
# → GitHub Settings → SSH and GPG keys → New GPG key
```


8. 脆弱性を見つけたら

Issue に書いてはいけない理由

 Issue は公開されている

脆弱性の詳細を Issue に書く = 世界中に公開

→ 修正前に攻撃者に悪用される

Security Report を使う

GitHub の Security Advisory 機能

Security → Report a vulnerability から非公開で報告できる

報告すべき内容

- 脆弱性の種類（XSS, SQLインジェクション 等）
- 再現手順（PoC）
- 影響範囲（Scope）
- 可能であれば修正案

流れ

非公開で報告 → 開発者と非公開で議論
→ 修正完了 → CVE 発行 → 公開

まとめ

テーマ	要点
Git の基本	commit / push / pull でチーム開発を安全に
ブランチ戦略	main への直 push 禁止・GitHub Flow
<code>.gitignore</code>	シークレットは絶対にpushしない
Branch Protection	ルールは機能で強制する
PR・レビュー	小さく出して・丁寧に議論する
Sign Commit	GPG署名でなりすましを防ぐ
脆弱性報告	Issue ではなく Security Report へ

参考リンク

- [GitHub Docs](#)
- [GitHub Flow](#)
- [About secret scanning](#)
- [Managing GPG keys](#)
- [Private security advisories](#)

Q&A

ご質問はありますか？